Invasive Species Food Web Analysis — Fernando de Noronha

Diet reconstruction, interaction network, indirect effects, and management scenarios

Micheletti and Gaiotto et al. 2026

15 July 2026

Contents

Overview

This notebook reconstructs the predator–prey food web for four invasive/focal consumers on Fernando de Noronha — feral cats (*Felis catus*), rats (*Rattus* spp.), tegu lizards (*Salvator merianae*), and cane toads (*Rhinella jimi*) — and evaluates how their removal (individually or in combination) propagates through the food web via direct and indirect effects.

The workflow has four main stages:

1. **Taxonomic reconstruction** — harmonize raw diet records into a common set of prey categories and calculate the Index of Relative Importance (IRI) for each predator's diet.
2. **Network construction** — combine diet composition (IRI) with predator density and sampling effort to build top-down (predator → prey) and bottom-up (prey → predator) interaction matrices, and visualize the resulting food web.

3. **Indirect effects** — use a total-effects matrix formalism (`indirect()` / `calculate_effect_matrix()`) to propagate direct interactions into whole-network (direct + indirect) effects, together with a sensitivity analysis of the species' response-strength parameters.
4. **Management scenarios** — simulate partial to complete removal of one or more invasive predators (with and without diet redistribution among the remaining predators) and quantify the predicted consequences for two endemic species of particular conservation concern (the skink *Trachylepis atlantica* and the amphisbaenian *Amphisbaena ridleyi*) and for the native rodent *Kerodon rupestris*.

All figures and tables are written to the `outputs/` folder at the end of the notebook.

Part I — Taxonomic reconstruction

1. Load packages

We use `Require::Require()` so that packages are installed automatically if missing, which keeps the analysis reproducible across machines/collaborators.

```
if (!require("Require")) {
  install.packages("Require")
}
library("Require")

Require::Require("readxl")    # read the raw Excel workbook
Require::Require("data.table") # fast, concise data wrangling
Require::Require("igraph")    # graph object underlying the food web
Require::Require("ggraph")    # ggplot2-style network plotting
Require::Require("ggplot2")   # all other plots (heatmaps, bar charts, etc.)
Require::Require("reshape2") # melt() for heatmap-ready long format
```

2. Read input workbook and create output directory

All raw data live in a single Excel workbook (`data/foodwebData.xlsx`), with one sheet per predator (`cats`, `rats`, `tegu`, `toads`) plus supporting sheets (`mapping`, `species_parameters`, `weighting`).

```
input_file <- "data/foodwebData.xlsx"
sheet_names <- excel_sheets(input_file)

sheets <- lapply(sheet_names, function(x) {
  as.data.table(read_excel(input_file, sheet = x))
})
names(sheets) <- sheet_names

mapping            <- copy(sheets$mapping)
species_parameters  <- copy(sheets$species_parameters)
weighting          <- copy(sheets$weighting)
cats  <- copy(sheets$cats)
rats  <- copy(sheets$rats)
tegu  <- copy(sheets$tegu)
```

```r
toads <- copy(sheets$toads)

# Toads require one extra derived column: estimated prey volume
# (mean volume per item x number of items), since toad diet data was
# recorded as counts + mean item volume rather than a single volume figure.
toads[, volume_ml := mean_volume_mL_V * count_N]

dir.create("outputs", showWarnings = FALSE)
```

3. Source helper functions

Custom functions for taxonomy harmonization, diet aggregation, IRI calculation, the total-effects formalism, and management-scenario simulation all live in `functions/helpers.R`.

```r
source("functions/helpers.R")
```

4. Harmonize prey names

Raw diet items are recorded at inconsistent taxonomic resolutions across predators (e.g. species-level vs. broad categories). `harmonize_taxonomy()` maps every raw prey name onto a common set of ecological categories, using the `mapping` sheet as a lookup table.

Notes for co-authors editing the mapping table:

- To **remove** a diet item entirely, set `Keep = FALSE` for that row in the `mapping` sheet and re-run the pipeline from the start.
- To **change how items are grouped**, add a new column to `mapping` and point `finalNaming` at it (we currently use the `Ecological_Group` column, referenced here as `"Ecological"`).
- **Caution:** the same raw name can appear in multiple candidate groupings. If you need to exclude an item from just one grouping scheme, add a slightly renamed duplicate row for that scheme (with `Keep = TRUE`) rather than editing the shared name, to avoid unintentionally removing it from other groupings too.

```r
cats  <- harmonize_taxonomy(cats,  mapping, finalNaming = "Ecological")
rats  <- harmonize_taxonomy(rats,  mapping, finalNaming = "Ecological")
tegu  <- harmonize_taxonomy(tegu,  mapping, finalNaming = "Ecological")
toads <- harmonize_taxonomy(toads, mapping, finalNaming = "Ecological")
```

5. Aggregate prey types

Diet records are aggregated per sample (where sample-level data exist) so that later frequency/occurrence-based metrics (e.g. IRI) are calculated correctly rather than being inflated by multiple rows per individual.

```r
cats  <- aggregate_raw_diet(cats,  predator = "cats")
rats  <- aggregate_raw_diet(rats,  predator = "rats")
tegu  <- aggregate_raw_diet(tegu,  predator = "tegu")
toads <- aggregate_raw_diet(toads, predator = "toads")
```

6. Calculate Index of Relative Importance (IRI) for all predators

IRI combines the numeric, frequency, and volumetric/weight contribution of each prey category to a predator's diet into a single importance score, expressed here as `percent_IRI`. Toads use a fixed stomach sample size (`n_stomachs = 137`) since their raw data were already summarized; the other three predators use per-individual raw records (`rawData = TRUE`).

```
toads_iri <- calculate_iri(diet_data = toads, n_stomachs = 137, rawData = FALSE)
cats_iri  <- calculate_iri(diet_data = cats,  rawData = TRUE)
rats_iri  <- calculate_iri(diet_data = rats,  rawData = TRUE)
tegu_iri  <- calculate_iri(diet_data = tegu,  rawData = TRUE)
```

7. Combine predator diets into a single diet-network table

```
cats_iri[,  predator := "Felis_catus"]
rats_iri[,  predator := "Rattus_spp"]
tegu_iri[,  predator := "Salvator_merianae"]
toads_iri[, predator := "Rhinella"]

diet_network <- rbindlist(
  list(cats_iri, rats_iri, tegu_iri, toads_iri),
  use.names = TRUE
)
```

Part III — Final interaction table

We keep only the columns needed downstream (predator, prey, and interaction strength = %IRI), and sort so the strongest diet links appear first within each predator — useful for a quick sanity check of the diet data.

```
diet_network <- diet_network[, .(
  predator,
  prey,
  interaction_strength = percent_IRI
)]

setorder(diet_network, predator, -interaction_strength)
```

8. Density-weighted predator consumption

IRI describes diet **composition** (i.e., relative importance of prey within a predator's diet) and is used below for **prey → predator** (bottom-up) effects. To estimate actual **predation pressure on the landscape** (top-down effects), we additionally need the number of individuals of each prey consumed per predator, corrected for sampling effort and then weighted by predator population density — since a predator with high per-capita consumption but very low density may exert weaker top-down pressure than a less selective but much denser predator.

Density and sample-size values are taken from published estimates for Fernando de Noronha and are documented inline below with citations.

```
## Number of stomachs/samples analysed per predator
samples <- c(
  Felis_catus       = 78,  # cats (n = 78), Gaiotto et al. 2020
  Salvator_merianae = 22,  # tegu (n = 22), Gaiotto et al. 2020
  Rattus_spp        = 10,  # rats (n = 10), Gaiotto et al. 2020
  Rhinella          = 137  # individual toad samples collected for this study
)

## Predator densities (individuals / ha)
densities <- c(
  Felis_catus       = 0.71,  # feral cats, Dias et al. 2017
  Salvator_merianae = 3.98,  # tegu, Abrahão et al. 2019
  Rattus_spp        = 37,    # rats, Russell et al. 2018
  Rhinella          = 10.35  # toads, extrapolated from Solomon Islands, Pikacha et al. 2015
)
```

```
predator_consumption <- rbindlist(list(
  cats_iri[,  .(predator = "Felis_catus",       prey, N)],
  rats_iri[,  .(predator = "Rattus_spp",        prey, N)],
  tegu_iri[,  .(predator = "Salvator_merianae", prey, N)],
  toads_iri[, .(predator = "Rhinella",          prey, N)]
))

## Build prey x predator consumption matrix (counts)
predator_consumption <- as.data.table(dcast(
  predator_consumption,
  prey ~ predator,
  value.var = "N",
  fill = 0
))

## Convert raw counts to per-predator (per-sample) consumption rate
predator_consumption[, names(samples) := Map(`/`, .SD, samples),
                     .SDcols = names(samples)]

## Weight per-predator consumption by predator density, giving an estimate
## of population-level predation pressure per unit area
predator_consumption[, names(densities) := Map(`*`, .SD, densities),
                     .SDcols = names(densities)]

## Keep prey as the first column for readability
setcolorder(predator_consumption, c("prey", names(samples)))
```

8b. Build the predator → prey interaction matrix

All species that appear either as prey or as a predator become nodes of the food web (`network_species`). We then build a square matrix where rows are prey and columns are predators, filled with density-weighted consumption values (i.e., **top-down pressure**).

```r
network_species <- sort(unique(c(
  predator_consumption$prey,
  predator_consumption |> names() |> setdiff("prey")
)))

predator_prey_matrix <- matrix(
  0,
  nrow = length(network_species),
  ncol = length(network_species),
  dimnames = list(network_species, network_species)
)

for (i in seq_len(nrow(predator_consumption))) {
  prey <- predator_consumption$prey[i]
  for (predator in names(samples)) {
    predator_prey_matrix[prey, predator] <- predator_consumption[i, get(predator)]
  }
}

# Sanity checks: every prey / predator name used to fill the matrix must
# actually exist among its row/column names, or the fill step above would
# have silently failed.
stopifnot(all(predator_consumption[, prey] %in% rownames(predator_prey_matrix)))
stopifnot(names(samples) %in% colnames(predator_prey_matrix))
```

9. Normalized resource-dependency matrix (bottom-up effects)

This matrix captures **prey → predator** dependency: how much each predator relies on each prey item, expressed as a proportion of that predator's total diet (i.e., IRI values re-scaled to sum to 1 within each predator). Rows are consumers (predators), columns are resources (prey) — this reproduces the orientation of `t(psiri)` in the original model this pipeline is based on.

```r
resource_dependency_matrix <- matrix(
  0,
  nrow = length(network_species),
  ncol = length(network_species),
  dimnames = list(network_species, network_species)
)

## Relative diet composition per predator (sums to 1 within each predator)
diet_network[, dependency := interaction_strength / sum(interaction_strength),
             by = predator]

for (i in seq_len(nrow(diet_network))) {
  resource_dependency_matrix[
    diet_network$predator[i],
    diet_network$prey[i]
  ] <- diet_network$dependency[i]
}
```

10. Species response parameters

`species_parameters` (from the workbook) holds each species' node label, trophic group (used for plotting), and its **response strength** — a species-specific scalar describing how strongly a species reacts to pressure from the network, used in the effect-propagation model below.

```
## Every network species must have matching parameters, or the model
## cannot be evaluated for it.
missing_species <- setdiff(network_species, species_parameters$species)

if (length(missing_species) > 0) {
  stop("Missing parameters for: ", paste(missing_species, collapse = ", "))
}

## Order parameters to match the network's species order
setkey(species_parameters, species)
species_parameters <- species_parameters[network_species]

response_strength <- species_parameters$response_strength
names(response_strength) <- species_parameters$species
```

11. Assemble food-web model inputs

```
foodweb_model <- list(
  species                    = network_species,
  predator_prey_matrix       = predator_prey_matrix,       # top-down
  resource_dependency_matrix = resource_dependency_matrix, # bottom-up
  response_strength          = response_strength,
  predator_densities         = densities,
  predator_samples           = samples
)
```

Part II — Food-web visualization

14. Visualize the food web

Nodes carry a display `label` and `trophic_group` (for point colour); edges are the predator–prey diet links from `diet_network`, with edge width mapped to interaction strength (%IRI).

```
nodes <- data.table(name = network_species)

nodes <- merge(
  nodes,
  species_parameters[, .(species, label, trophic_group)],
  by.x = "name",
  by.y = "species",
  all.x = TRUE
```

```
)

edges <- copy(diet_network)
setnames(edges, c("prey", "predator"), c("from", "to"))

foodweb_graph <- graph_from_data_frame(
  d = edges,
  vertices = nodes,
  directed = TRUE
)
```

```
foodweb_plot <- ggraph(foodweb_graph, layout = "stress") +

  geom_edge_link(
    aes(width = interaction_strength),
    alpha = 0.7,
    colour = "grey40",
    end_cap = circle(5, "mm"),
    start_cap = circle(5, "mm"),
    arrow = arrow(length = unit(3.5, "mm"), type = "closed")
  ) +

  geom_node_point(
    aes(fill = trophic_group),
    shape = 21,
    size = 6,
    colour = "black",
    stroke = 0.4
  ) +

  geom_node_text(
    aes(label = label),
    repel = TRUE,
    size = 3.8,
    point.padding = unit(0.5, "lines"),
    box.padding = unit(0.7, "lines")
  ) +

  scale_edge_width(range = c(0.2, 2.8)) +

  # base_family = "sans" avoids "font family not found in Windows font
  # database" warnings from theme_graph() on Windows machines.
  theme_graph(base_family = "sans")

foodweb_plot
```

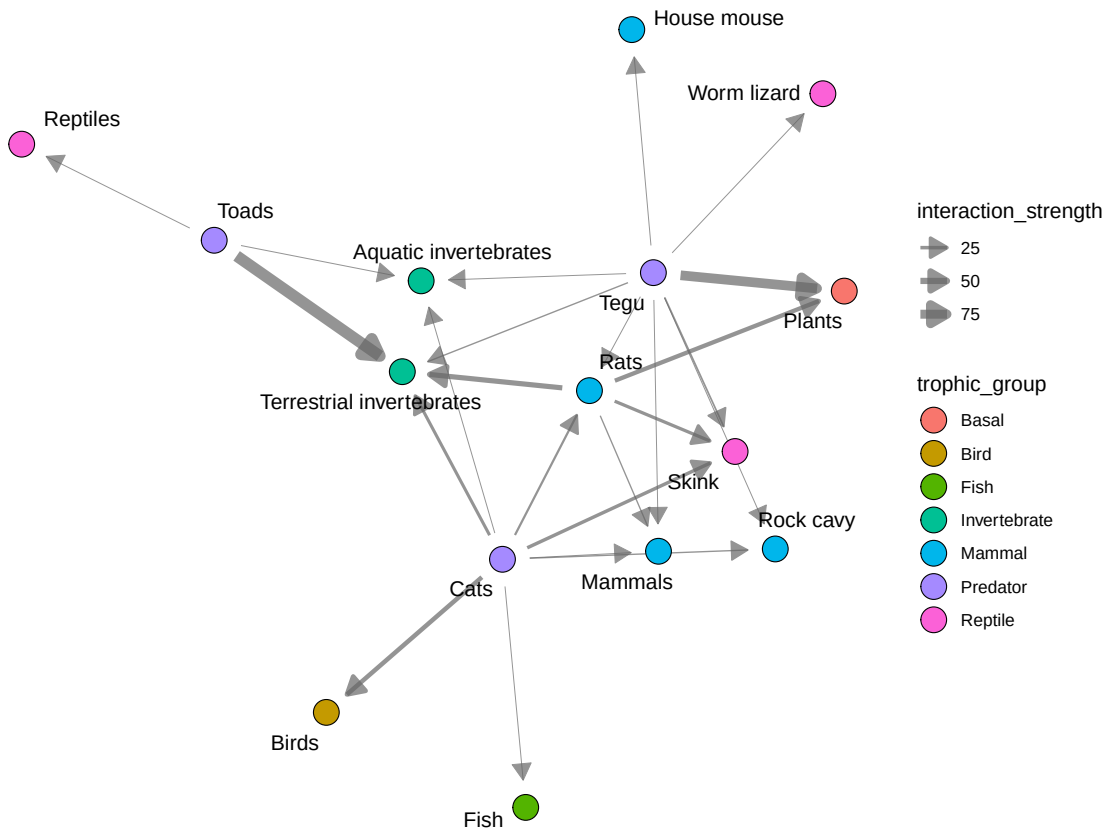

Figure 1: Food web of the four focal predators and their prey on Fernando de Noronha. Edge width = interaction strength (%IRI); node colour = trophic group.

Part IV — Direct and indirect effects

15. Build the direct-interaction matrices

We now assemble the two matrices that feed the total-effects model: top-down predation effects and bottom-up resource-dependency effects, and confirm they share identical species ordering (required for matrix algebra downstream).

```
predation_matrix    <- copy(predator_prey_matrix)       # top-down
resource_dep_matrix <- copy(resource_dependency_matrix) # bottom-up
response_strength   <- response_strength

stopifnot(identical(rownames(predation_matrix), rownames(resource_dep_matrix)))
stopifnot(identical(colnames(predation_matrix), colnames(resource_dep_matrix)))
stopifnot(identical(rownames(predation_matrix), names(response_strength)))
```

16. Build the combined direct-effect matrix

Species are reordered into a fixed, ecologically meaningful sequence (`network_order`, native/endemic species first, then invasive predators) so that all downstream matrices and plots use a consistent, readable ordering.

```
predator_effect_matrix <- copy(predator_prey_matrix) # analogous to Bw (top-down)

# psiri (the original diet-composition matrix) had prey columns first and
# predators last; we standardize row/column order here to network_order so
# top-down and bottom-up matrices line up correctly for addition below.
network_order <- c(
  "Amphisbaena_ridleyi",
  "Aquatic_invertebrates",
  "Birds",
  "Fish",
  "Kerodon_rupestris",
  "Mammals",
  "Mus_musculus",
  "Plants",
  "Reptiles",
  "Terrestrial_invertebrates",
  "Trachylepis_atlantica",
  "Rhinella",
  "Rattus_spp",
  "Salvator_merianae",
  "Felis_catus"
)

stopifnot(setequal(network_order, rownames(predator_prey_matrix)))
stopifnot(setequal(network_order, rownames(resource_dependency_matrix)))

predator_effect_matrix     <- predator_effect_matrix[network_order, network_order]
resource_dependency_matrix <- resource_dependency_matrix[network_order, network_order]
response_strength          <- response_strength[network_order]
```

```
## Bottom-up direct effects, analogous to M = 0.1 * t(psiri) in the
## original model (psiri was expressed in %).
## IMPORTANT: resource_dependency_matrix here is ALREADY transposed
## relative to psiri, i.e. t(psiri) ~ resource_dependency_matrix / 100.
## We therefore multiply by 100 (to undo the earlier /100 normalization)
## and then by 0.1 (the model's bottom-up scaling constant), WITHOUT an
## additional transpose.
resource_effect_matrix <- 0.1 * 100 * resource_dependency_matrix
```

```
## Optional relative weighting of top-down vs. bottom-up processes.
## Currently both are unweighted (weight = 1); the commented-out lines
## show how weights could instead be read from the workbook's
## `weighting` sheet if we later want to explore that sensitivity.

# predation_weight <- weighting[parameter == "predation_weight", value]
# resource_weight  <- weighting[parameter == "resource_weight", value]
predation_weight <- 1
resource_weight  <- 1

# Combined direct-effect matrix: M <- M + Bw
effects_Matrix <- (predation_weight * predator_effect_matrix) +
                  (resource_weight  * resource_effect_matrix)
```

17. Calculate total (direct + indirect) effects

`calculate_effect_matrix()` implements the total-effects matrix formalism (built around the `indirect()` function), propagating each species' response strength through the direct-effect matrix to yield direct and total (direct + indirect) effect matrices for every species pair.

```
effect_results <- calculate_effect_matrix(
  mat = effects_Matrix,
  R = response_strength
)
```

```
## Maximum |W|: 1
## Maximum |PW|: 0.785999
```

18. Visualize network effects

Diagonal values (self-effects) are set to `NA` before plotting, since they are not ecologically meaningful for a species-pair heatmap.

```
total_effect_matrix <- effect_results$total_effect_matrix
diag(total_effect_matrix) <- NA

direct_effect_matrix <- effect_results$direct_effect_matrix
diag(direct_effect_matrix) <- NA
```

Positive vs. negative effects and species rankings

We split total effects into their positive and negative components and sum across each acting species, to identify which species have the strongest net-positive vs. net-negative influence on the rest of the network.

```
positive_effect_matrix <- pmax(total_effect_matrix, 0)
negative_effect_matrix <- pmin(total_effect_matrix, 0)

positive_effect_out <- colSums(positive_effect_matrix, na.rm = TRUE)
negative_effect_out <- colSums(negative_effect_matrix, na.rm = TRUE)

positive_effect_ranking <- data.frame(
  species = names(sort(positive_effect_out, decreasing = TRUE)),
  total_positive_effect = sort(positive_effect_out, decreasing = TRUE)
)

negative_effect_ranking <- data.frame(
  species = names(sort(negative_effect_out)),
  total_negative_effect = sort(negative_effect_out)
)
```

Heatmaps of direct and total effects

```
total_effect_heatmap <- ggplot(
  melt(total_effect_matrix),
  aes(Var2, factor(Var1, levels = rev(rownames(total_effect_matrix))), fill = value)
) +
  geom_tile(color = "grey90") +
  scale_fill_gradient2(
    low = "#B2182B", mid = "white", high = "#2166AC",
    midpoint = 0, na.value = "grey90"
  ) +
  coord_fixed() +
  labs(x = "Acting species", y = "Affected species", fill = "Total effect") +
  theme_bw() +
  theme(panel.grid = element_blank(), axis.text.x = element_text(angle = 45, hjust = 1))

direct_effect_heatmap <- ggplot(
  melt(direct_effect_matrix),
  aes(Var2, factor(Var1, levels = rev(rownames(direct_effect_matrix))), fill = value)
) +
  geom_tile(color = "grey90") +
  scale_fill_gradient2(
    low = "#B2182B", mid = "white", high = "#2166AC",
    midpoint = 0, na.value = "grey90"
  ) +
  coord_fixed() +
  labs(x = "Acting species", y = "Affected species", fill = "Direct effect") +
  theme_bw() +
  theme(panel.grid = element_blank(), axis.text.x = element_text(angle = 45, hjust = 1))

total_effect_heatmap
```

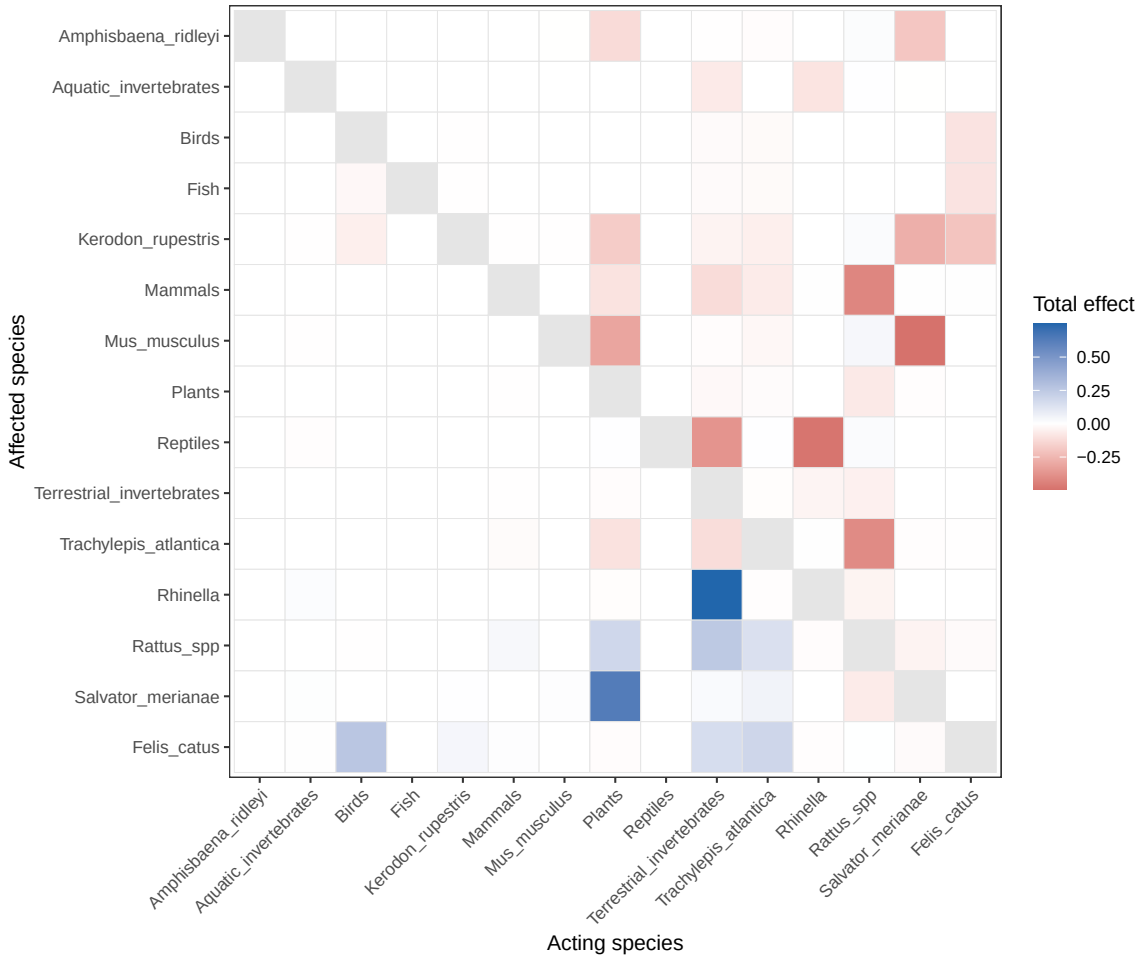

Figure 2: Direct vs. total (direct + indirect) effect matrices among network species. Blue = positive effect, red = negative effect.

```
direct_effect_heatmap
```

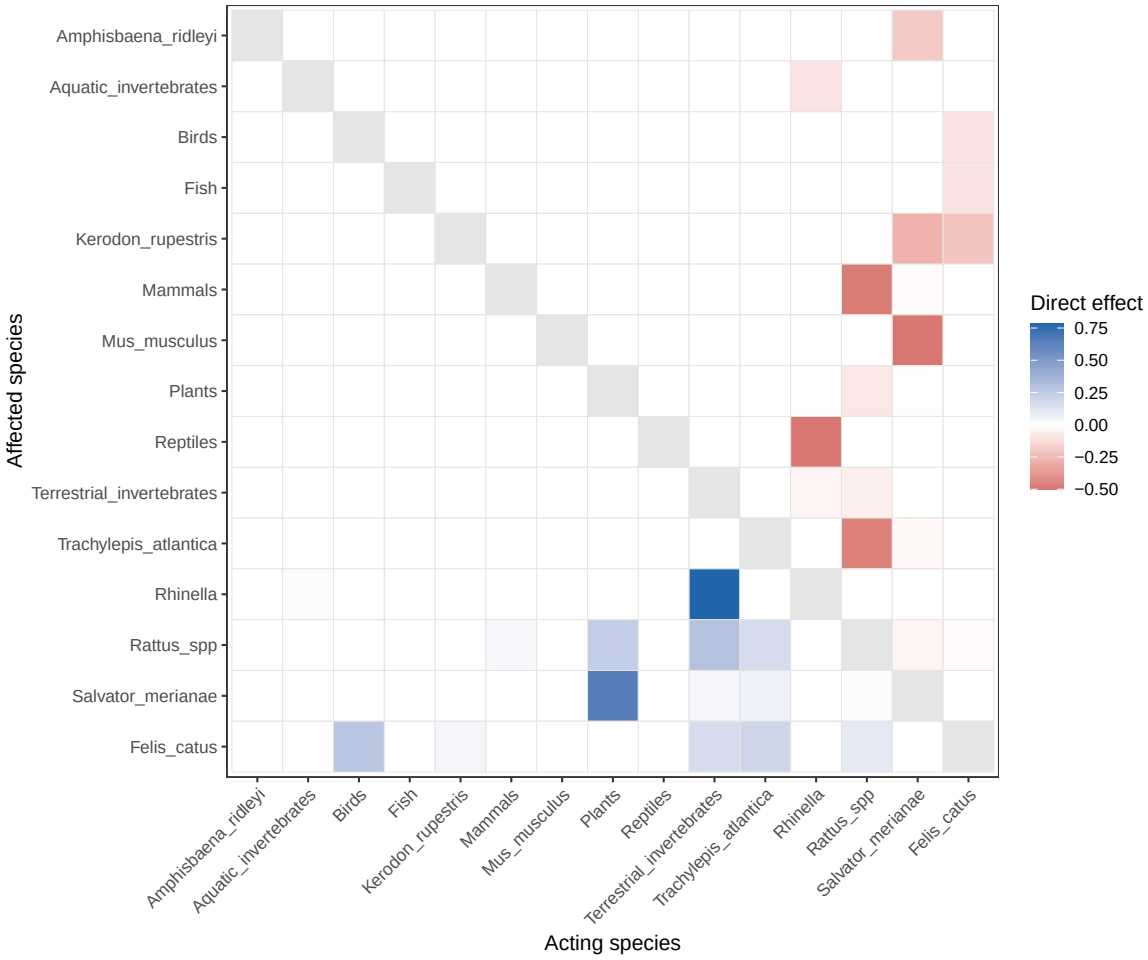

Figure 3: Direct vs. total (direct + indirect) effect matrices among network species. Blue = positive effect, red = negative effect.

Part V — Sensitivity analysis

We evaluate how robust the total-effects ranking is to uncertainty in the species response-strength parameter (`R`), by resampling `R` within ±20% of its point estimate (bounded to [0, 1]) across 5000 simulations, and comparing each simulation's ranking of species influence to the baseline ranking (Spearman correlation, top-1 species, and top-5 species set).

```
n_simulations <- 5000

baseline_effects <- effect_results$mean_total_effect
baseline_ranking <- names(sort(baseline_effects, decreasing = TRUE))
```

```r
spearman_correlations <- numeric(n_simulations)
top1_species <- character(n_simulations)
top5_species <- vector(mode = "list", length = n_simulations)
ranking_identical <- numeric(n_simulations)
```

```r
for (i in seq_len(n_simulations)) {

  ## Perturb response strengths by +/-20%, keeping values within [0, 1]
  response_strength_temp <- response_strength *
    runif(length(response_strength), min = 0.8, max = 1.2)

  response_strength_temp <- pmin(1, pmax(0, response_strength_temp))
  names(response_strength_temp) <- names(response_strength)
  response_strength_temp <- response_strength_temp[rownames(direct_effect_matrix)]

  ## Recalculate total effects with the perturbed response strengths
  effect_results_temp <- calculate_effect_matrix(
    mat = effects_Matrix,
    R   = response_strength_temp
  )

  temp_effects <- effect_results_temp$mean_total_effect

  ## Compare this simulation's ranking to the baseline ranking
  spearman_correlations[i] <- cor(baseline_effects, temp_effects, method = "spearman")

  ranking <- names(sort(temp_effects, decreasing = TRUE))
  top1_species[i] <- ranking[1]
  top5_species[[i]] <- ranking[1:5]

  ranking_identical[i] <- identical(ranking, baseline_ranking)
}
```

```
## Maximum |W|: 1
## Maximum |PW|: 0.9075652
## Maximum |W|: 1
## Maximum |PW|: 0.7697021
## Maximum |W|: 1
## Maximum |PW|: 0.7415465
## Maximum |W|: 1
## Maximum |PW|: 0.6985796
## Maximum |W|: 1
## Maximum |PW|: 0.7376841
## Maximum |W|: 1
## Maximum |PW|: 0.7576606
## Maximum |W|: 1
## Maximum |PW|: 0.7160373
## Maximum |W|: 1
## Maximum |PW|: 0.825932
## Maximum |W|: 1
## Maximum |PW|: 0.7503726
## Maximum |W|: 1
## Maximum |PW|: 0.7838546
```

```
## Maximum |W|: 1
## Maximum |PW|: 0.724502
## Maximum |W|: 1
## Maximum |PW|: 0.9406883
## Maximum |W|: 1
## Maximum |PW|: 0.822364
## Maximum |W|: 1
## Maximum |PW|: 0.8992574
## Maximum |W|: 1
## Maximum |PW|: 0.8364428
## Maximum |W|: 1
## Maximum |PW|: 0.7593989
## Maximum |W|: 1
## Maximum |PW|: 0.7983716
## Maximum |W|: 1
## Maximum |PW|: 0.7685757
## Maximum |W|: 1
## Maximum |PW|: 0.8726818
## Maximum |W|: 1
## Maximum |PW|: 0.9067159
## Maximum |W|: 1
## Maximum |PW|: 0.8299482
## Maximum |W|: 1
## Maximum |PW|: 0.8467453
## Maximum |W|: 1
## Maximum |PW|: 0.7985213
## Maximum |W|: 1
## Maximum |PW|: 0.676313
## Maximum |W|: 1
## Maximum |PW|: 0.7755507
## Maximum |W|: 1
## Maximum |PW|: 0.7782446
## Maximum |W|: 1
## Maximum |PW|: 0.9065991
## Maximum |W|: 1
## Maximum |PW|: 0.927818
## Maximum |W|: 1
## Maximum |PW|: 0.7682851
## Maximum |W|: 1
## Maximum |PW|: 0.8866973
## Maximum |W|: 1
## Maximum |PW|: 0.6872765
## Maximum |W|: 1
## Maximum |PW|: 0.6701538
```

Summary statistics

```
sensitivity_summary <- data.frame(
  statistic = c(
    "Mean Spearman correlation",
    "Median Spearman correlation",
    "Minimum Spearman correlation",
```

```
    "Maximum Spearman correlation",
    "Proportion of identical rankings"
  ),
  value = c(
    mean(spearman_correlations),
    median(spearman_correlations),
    min(spearman_correlations),
    max(spearman_correlations),
    mean(ranking_identical)
  )
)

sensitivity_summary
```

```
##                          statistic     value
## 1        Mean Spearman correlation 0.9774900
## 2      Median Spearman correlation 1.0000000
## 3     Minimum Spearman correlation 0.8928571
## 4     Maximum Spearman correlation 1.0000000
## 5 Proportion of identical rankings 0.5212000
```

Rank-1 and top-5 stability across simulations

```
top_rank_stability <- sort(prop.table(table(top1_species)), decreasing = TRUE)
top_rank_stability <- data.frame(
  species = names(top_rank_stability),
  proportion_top1 = as.numeric(top_rank_stability)
)

top5_stability <- sapply(names(baseline_effects), function(sp) {
  mean(sapply(top5_species, function(x) sp %in% x))
})
top5_stability <- sort(top5_stability, decreasing = TRUE)
top5_stability <- data.frame(
  species = names(top5_stability),
  proportion_top5 = top5_stability
)
```

```
response_strength_sensitivity_plot <- ggplot(
  data.frame(correlation = spearman_correlations), aes(correlation)
) +
  geom_histogram(bins = 30, fill = "grey70", colour = "black") +
  theme_classic() +
  xlab("Spearman correlation with baseline species ranking") +
  ylab("Number of simulations") +
  ggtitle("Robustness of species rankings to uncertainty in response strengths")

plot_top5_stability <- ggplot(
  top5_stability, aes(x = reorder(species, proportion_top5), y = proportion_top5)
) +
  geom_col(fill = "steelblue") +
```

```
  coord_flip() +
  scale_y_continuous(limits = c(0, 1), expand = c(0, 0)) +
  labs(
    title = "Top-5 ranking stability",
    subtitle = "Proportion of simulations in which each species ranked among the five most influential"
    x = NULL, y = "Proportion of simulations"
  ) +
  theme_bw()

response_strength_sensitivity_plot
```

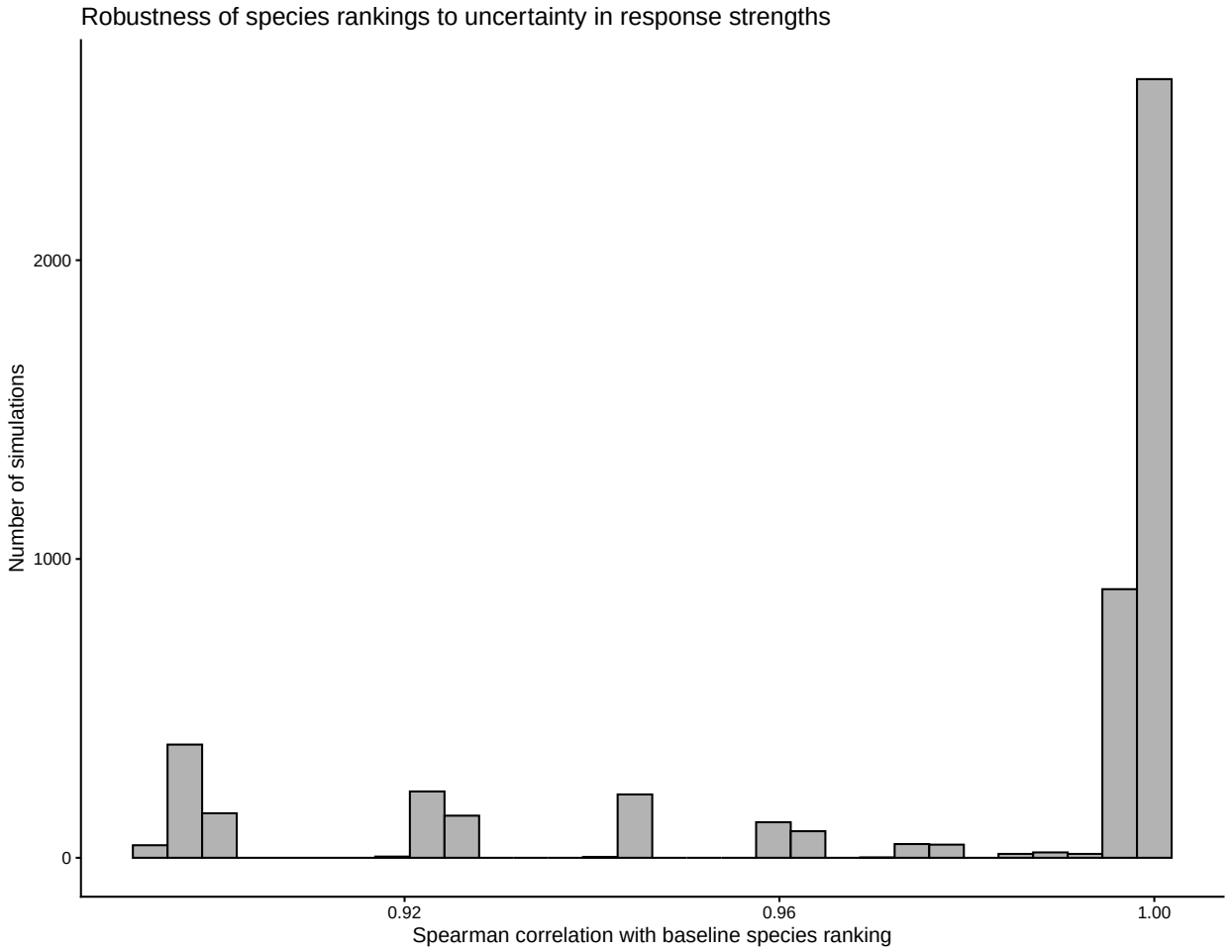

Figure 4: Left: distribution of Spearman correlations between baseline and perturbed species rankings across 5,000 simulations. Right: proportion of simulations in which each species ranked among the five most influential.

```
plot_top5_stability
```

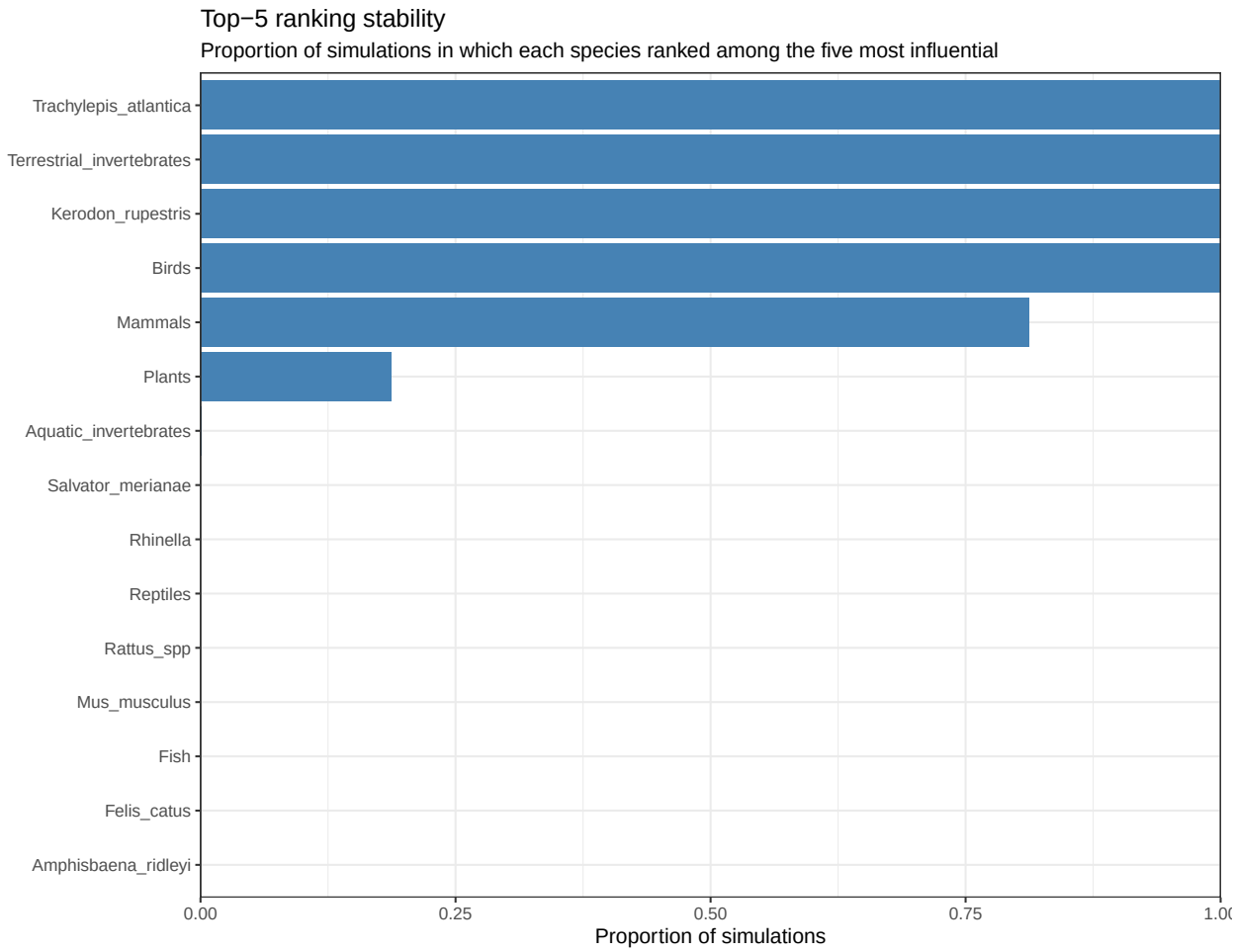

Figure 5: Left: distribution of Spearman correlations between baseline and perturbed species rankings across 5,000 simulations. Right: proportion of simulations in which each species ranked among the five most influential.

Part VI — Management scenarios

20. Baseline

```
baseline_results <- effect_results
baseline_total_effect_matrix <- baseline_results$total_effect_matrix
baseline_mean_total_effect <- baseline_results$mean_total_effect
```

Define management scenarios

We simulate:

- **Single-predator reductions** at five intensities (25%, 50%, 75%, 90%, ~100%) for each of the four focal predators.
- **Combined eradications** of two or more predators simultaneously (cats + rats; cats + tegu; all four invasive species).

```
management_scenarios <- list()

managed_predators <- c("Felis_catus", "Rattus_spp", "Salvator_merianae", "Rhinella")
management_levels <- c(0.25, 0.50, 0.75, 0.90, 0.9999)

for (predator in managed_predators) {
  for (level in management_levels) {
    scenario_name <- paste0(predator, "_", round(level * 100), "percent")
    management_scenarios[[scenario_name]] <- data.frame(
      source = predator,
      target = NA,
      reduction = level,
      stringsAsFactors = FALSE
    )
  }
}

management_scenarios[["Cats_and_Rats"]] <- data.frame(
  source = c("Felis_catus", "Rattus_spp"), target = NA, reduction = 0.9999
)

management_scenarios[["Cats_and_Tegu"]] <- data.frame(
  source = c("Felis_catus", "Salvator_merianae"), target = NA, reduction = 0.9999
)

management_scenarios[["All_Invasive"]] <- data.frame(
  source = c("Felis_catus", "Rattus_spp", "Salvator_merianae", "Rhinella"),
  target = NA, reduction = 0.9999
)
```

Run all scenarios

Each scenario is run twice: once with prey redistribution among remaining predators disabled (`redistribution = "none"`) and once with it enabled (`redistribution = "predator_diet"`, the

behavioural assumption that surviving predators shift their diet toward remaining resources). The `"predator_diet"` version (`management_results`) is what we use for the main figures; the `"none"` version is used only for the sensitivity comparison below.

```
management_results <- list()
for (name in names(management_scenarios)) {
  management_results[[name]] <- simulate_management_scenario(
    effect_matrix = effects_Matrix,
    response_strength = response_strength,
    modifications = management_scenarios[[name]],
    redistribution = "predator_diet"
  )
}
```

```
## Maximum |W|: 1
## Maximum |PW|: 0.7852497
## Maximum |W|: 1
## Maximum |PW|: 0.7845269
## Maximum |W|: 1
## Maximum |PW|: 0.7838293
## Maximum |W|: 1
## Maximum |PW|: 0.7834223
## Maximum |W|: 1
## Maximum |PW|: 0.7957591
## Maximum |W|: 1
## Maximum |PW|: 0.7971611
## Maximum |W|: 1
## Maximum |PW|: 0.7984204
## Maximum |W|: 1
## Maximum |PW|: 0.7989058
## Maximum |W|: 1
## Maximum |PW|: 0.7990761
## Maximum |W|: 1
## Maximum |PW|: 0.7991629
## Maximum |W|: 1
## Maximum |PW|: 0.7814131
## Maximum |W|: 1
## Maximum |PW|: 0.7771224
## Maximum |W|: 1
## Maximum |PW|: 0.7730991
## Maximum |W|: 1
## Maximum |PW|: 0.7708034
## Maximum |W|: 1
## Maximum |PW|: 0.7693206
## Maximum |W|: 1
## Maximum |PW|: 0.7859954
## Maximum |W|: 1
## Maximum |PW|: 0.7859882
## Maximum |W|: 1
## Maximum |PW|: 0.7859665
## Maximum |W|: 1
## Maximum |PW|: 0.7859014
## Maximum |W|: 1
## Maximum |PW|: 0.7414438
```

```
## Maximum |W|: 1
## Maximum |PW|: 0.7990967
## Maximum |W|: 1
## Maximum |PW|: 0.7620196
## Maximum |W|: 1
## Maximum |PW|: 0.7922476
```

```
management_results_none <- list()
for (name in names(management_scenarios)) {
  management_results_none[[name]] <- simulate_management_scenario(
    effect_matrix = effects_Matrix,
    response_strength = response_strength,
    modifications = management_scenarios[[name]],
    redistribution = "none"
  )
}
```

```
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
```

```
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
## Maximum |W|: 1
## Maximum |PW|: 0.785999
```

```
management_results_diet <- list()
for (name in names(management_scenarios)) {
  management_results_diet[[name]] <- simulate_management_scenario(
    effect_matrix = effects_Matrix,
    response_strength = response_strength,
    modifications = management_scenarios[[name]],
    redistribution = "predator_diet"
  )
}
```

```
## Maximum |W|: 1
## Maximum |PW|: 0.7852497
## Maximum |W|: 1
## Maximum |PW|: 0.7845269
## Maximum |W|: 1
## Maximum |PW|: 0.7838293
## Maximum |W|: 1
## Maximum |PW|: 0.7834223
## Maximum |W|: 1
## Maximum |PW|: 0.7957591
## Maximum |W|: 1
## Maximum |PW|: 0.7971611
## Maximum |W|: 1
## Maximum |PW|: 0.7984204
## Maximum |W|: 1
## Maximum |PW|: 0.7989058
## Maximum |W|: 1
## Maximum |PW|: 0.7990761
## Maximum |W|: 1
## Maximum |PW|: 0.7991629
## Maximum |W|: 1
## Maximum |PW|: 0.7814131
## Maximum |W|: 1
## Maximum |PW|: 0.7771224
## Maximum |W|: 1
## Maximum |PW|: 0.7730991
## Maximum |W|: 1
## Maximum |PW|: 0.7708034
## Maximum |W|: 1
## Maximum |PW|: 0.7693206
## Maximum |W|: 1
## Maximum |PW|: 0.7859954
## Maximum |W|: 1
## Maximum |PW|: 0.7859882
```

```
## Maximum |W|: 1
## Maximum |PW|: 0.7859665
## Maximum |W|: 1
## Maximum |PW|: 0.7859014
## Maximum |W|: 1
## Maximum |PW|: 0.7414438
## Maximum |W|: 1
## Maximum |PW|: 0.7990967
## Maximum |W|: 1
## Maximum |PW|: 0.7620196
## Maximum |W|: 1
## Maximum |PW|: 0.7922476
```

Sensitivity to the diet-redistribution assumption (focus: *Trachylepis atlantica*)

Because the behavioural assumption of prey redistribution among surviving predators could plausibly affect predictions for the endemic skink *Trachylepis atlantica*, we compare its predicted response with vs. without redistribution across all scenarios.

```
comparison_trachylepis <- data.frame(
  scenario = names(management_scenarios),
  no_redistribution = sapply(
    management_results_none,
    function(x) x$delta_mean_total_effect["Trachylepis_atlantica"]
  ),
  predator_redistribution = sapply(
    management_results_diet,
    function(x) x$delta_mean_total_effect["Trachylepis_atlantica"]
  )
)

comparison_trachylepis$difference <-
  comparison_trachylepis$predator_redistribution -
  comparison_trachylepis$no_redistribution
```

```
comparison_summary <- data.frame(
  scenario = names(management_scenarios),

  rank_correlation = sapply(names(management_scenarios), function(name) {
    cor(
      management_results_none[[name]]$delta_mean_total_effect,
      management_results_diet[[name]]$delta_mean_total_effect,
      method = "spearman"
    )
  }),

  mean_absolute_difference = sapply(names(management_scenarios), function(name) {
    mean(abs(
      management_results_none[[name]]$delta_mean_total_effect -
        management_results_diet[[name]]$delta_mean_total_effect
    ))
  }),
```

```
  max_absolute_difference = sapply(names(management_scenarios), function(name) {
    max(abs(
      management_results_none[[name]]$delta_mean_total_effect -
        management_results_diet[[name]]$delta_mean_total_effect
    ))
  })
)

comparison_summary
```

```
##                                                scenario rank_correlation
## Felis_catus_25percent                Felis_catus_25percent        0.4642857
## Felis_catus_50percent                Felis_catus_50percent        0.4535714
## Felis_catus_75percent                Felis_catus_75percent        0.4178571
## Felis_catus_90percent                Felis_catus_90percent        0.4178571
## Felis_catus_100percent              Felis_catus_100percent        0.4178571
## Rattus_spp_25percent                  Rattus_spp_25percent       -0.1250000
## Rattus_spp_50percent                  Rattus_spp_50percent        0.4107143
## Rattus_spp_75percent                  Rattus_spp_75percent        0.5571429
## Rattus_spp_90percent                  Rattus_spp_90percent        0.9357143
## Rattus_spp_100percent                Rattus_spp_100percent        0.9857143
## Salvator_merianae_25percent   Salvator_merianae_25percent        0.9250000
## Salvator_merianae_50percent   Salvator_merianae_50percent        0.9321429
## Salvator_merianae_75percent   Salvator_merianae_75percent        0.9642857
## Salvator_merianae_90percent   Salvator_merianae_90percent        0.9785714
## Salvator_merianae_100percent Salvator_merianae_100percent        0.9392857
## Rhinella_25percent                    Rhinella_25percent        0.3214286
## Rhinella_50percent                    Rhinella_50percent        0.6178571
## Rhinella_75percent                    Rhinella_75percent        0.6321429
## Rhinella_90percent                    Rhinella_90percent        0.6642857
## Rhinella_100percent                  Rhinella_100percent        0.7642857
## Cats_and_Rats                              Cats_and_Rats        0.6607143
## Cats_and_Tegu                              Cats_and_Tegu        0.6357143
## All_Invasive                                All_Invasive       -0.5392857
##                              mean_absolute_difference max_absolute_difference
## Felis_catus_25percent                      0.0003227476            0.0025615532
## Felis_catus_50percent                      0.0008542623            0.0067511092
## Felis_catus_75percent                      0.0018139353            0.0142975117
## Felis_catus_90percent                      0.0028176134            0.0221889864
## Felis_catus_100percent                     0.0038324612            0.0301727759
## Rattus_spp_25percent                       0.0016493405            0.0078045302
## Rattus_spp_50percent                       0.0021273086            0.0101058649
## Rattus_spp_75percent                       0.0023476363            0.0110904977
## Rattus_spp_90percent                       0.0025288162            0.0108546960
## Rattus_spp_100percent                      0.0032926510            0.0141886455
## Salvator_merianae_25percent                0.0001030100            0.0003611447
## Salvator_merianae_50percent                0.0002090701            0.0006486416
## Salvator_merianae_75percent                0.0003608150            0.0010536777
## Salvator_merianae_90percent                0.0006686961            0.0032436332
## Salvator_merianae_100percent               0.0042008928            0.0289658416
## Rhinella_25percent                         0.0022705069            0.0084778909
## Rhinella_50percent                         0.0028647243            0.0106618597
## Rhinella_75percent                         0.0031211541            0.0115289592
```

```
## Rhinella_90percent                0.0031988217          0.0116649058
## Rhinella_100percent               0.0035049582          0.0104002487
## Cats_and_Rats                     0.0082908521          0.0327246368
## Cats_and_Tegu                     0.0051868286          0.0211345965
## All_Invasive                      0.0043099751          0.0163390539
```

```
comparison_plot <- ggplot(
  comparison_trachylepis, aes(no_redistribution, predator_redistribution)
) +
  geom_abline(slope = 1, intercept = 0, linetype = 2) +
  geom_point(size = 3) +
  geom_text(aes(label = scenario), hjust = -0.1, size = 3) +
  theme_bw() +
  labs(
    x = "Without predator redistribution",
    y = "With predator redistribution",
    title = "Sensitivity of Trachylepis response to behavioural assumptions"
  )

comparison_plot
```

Common scales across management plots

To allow visual comparison across scenarios, we compute shared axis/colour limits based on the largest change observed in *any* scenario.

```
max_species_change <- max(sapply(
  management_results, function(x) max(abs(x$delta_mean_total_effect), na.rm = TRUE)
))

max_matrix_change <- max(sapply(
  management_results, function(x) max(abs(x$delta_total_effect_matrix), na.rm = TRUE)
))
```

Scenario x species response matrix

```
management_species_matrix <- do.call(rbind, lapply(
  management_results, function(x) x$delta_mean_total_effect
))
rownames(management_species_matrix) <- names(management_results)

management_species_matrix <- management_species_matrix[
  , names(sort(baseline_effects, decreasing = TRUE)), drop = FALSE
]
```

Summary tables

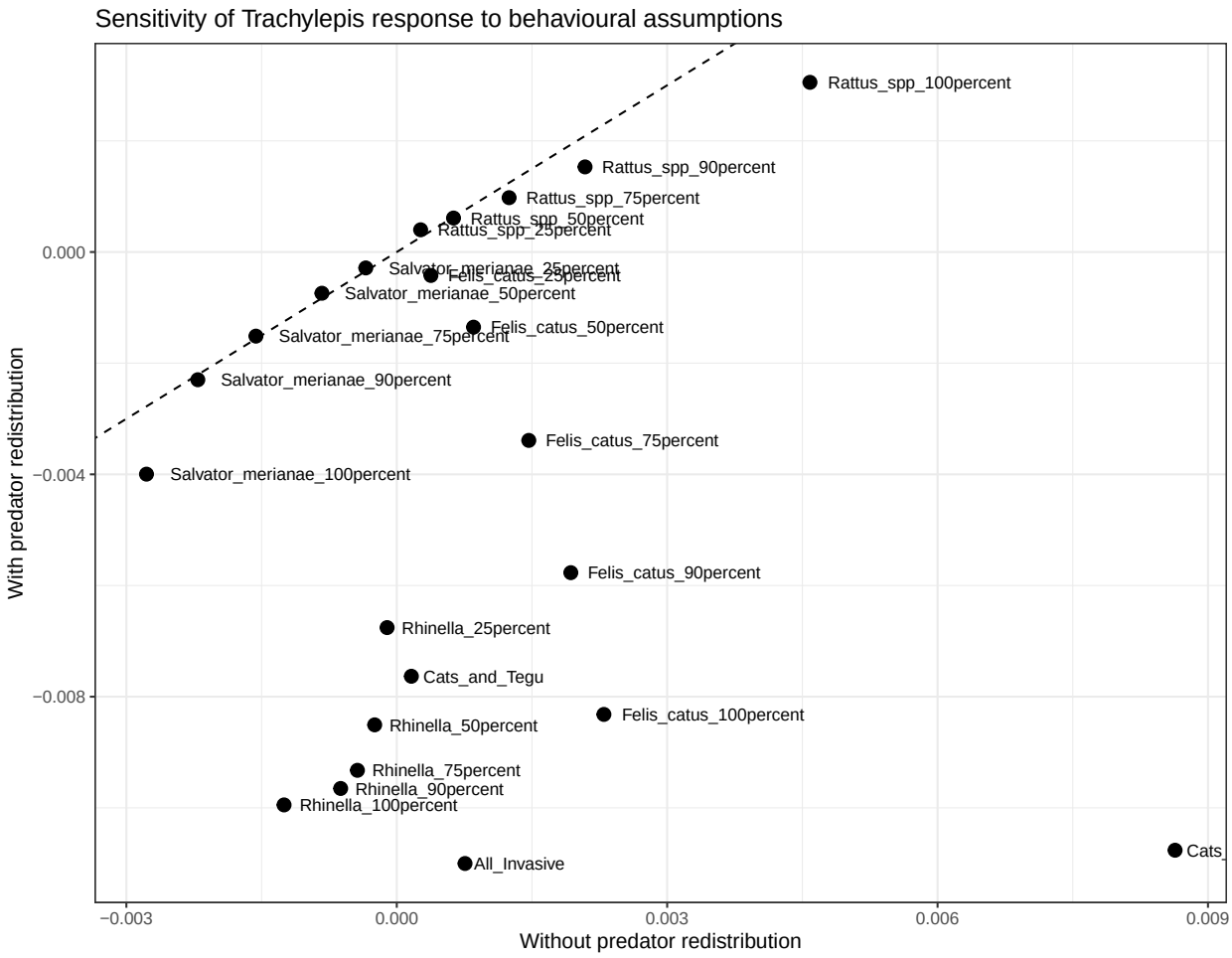

Figure 6: Predicted change in total effect on Trachylepis atlantica under each management scenario, with vs. without diet redistribution among surviving predators. Points near the dashed 1:1 line indicate the redistribution assumption has little influence on this species' predicted response.

```r
management_summary <- do.call(rbind, lapply(names(management_results), function(name) {
  data.frame(
    scenario = name,
    species = names(management_results[[name]]$delta_mean_total_effect),
    delta_mean_total_effect = as.numeric(management_results[[name]]$delta_mean_total_effect),
    stringsAsFactors = FALSE
  )
}))

management_sign_changes <- data.frame(
  scenario = names(management_results),
  sign_changes = sapply(management_results, function(x) x$sign_changes)
)
```

Responses of endemic / focal native species

We track three species of particular conservation interest: the two endemic reptiles (*Trachylepis atlantica*, *Amphisbaena ridleyi*) and the native rodent *Kerodon rupestris*.

```r
endemic_species <- c("Trachylepis_atlantica", "Amphisbaena_ridleyi", "Kerodon_rupestris")

management_endemic_species <- do.call(rbind, lapply(names(management_results), function(name) {
  data.frame(
    scenario = name,
    species = endemic_species,
    delta_mean_total_effect =
      management_results[[name]]$delta_mean_total_effect[endemic_species],
    stringsAsFactors = FALSE
  )
}))
```

Store per-scenario matrices

```r
management_direct_effect_matrices <- list()
management_total_effect_matrices <- list()
management_delta_effect_matrices <- list()

for (name in names(management_results)) {
  management_direct_effect_matrices[[name]] <- management_results[[name]]$direct_effect_matrix
  management_total_effect_matrices[[name]]  <- management_results[[name]]$total_effect_matrix
  management_delta_effect_matrices[[name]]  <- management_results[[name]]$delta_total_effect_matrix
}

max_delta_effect <- max(abs(unlist(management_delta_effect_matrices)), na.rm = TRUE)
```

Overall management comparison table

For each scenario we summarize: the mean network-wide change, how many species responded positively/negatively (excluding the managed species themselves), the number of sign changes relative to baseline, and the specific predicted change for each of the three focal native/endemic species.

```r
management_summary2 <- data.frame(
  scenario = names(management_results),

  mean_change = sapply(management_results, function(x) mean(x$delta_mean_total_effect)),

  positive_species = sapply(management_results, function(x) sum(x$delta_mean_total_effect > 0)),

  negative_species = sapply(names(management_results), function(name) {
    delta <- management_results[[name]]$delta_mean_total_effect
    managed_species <- management_scenarios[[name]]$source
    delta <- delta[!(names(delta) %in% managed_species)]
    sum(delta < 0)
  }),

  sign_changes = sapply(management_results, function(x) x$sign_changes)
)

management_summary2$Trachylepis <- sapply(
  management_results, function(x) x$delta_mean_total_effect["Trachylepis_atlantica"]
)
management_summary2$Amphisbaena <- sapply(
  management_results, function(x) x$delta_mean_total_effect["Amphisbaena_ridleyi"]
)
management_summary2$Kerodon <- sapply(
  management_results, function(x) x$delta_mean_total_effect["Kerodon_rupestris"]
)

management_summary2
```

```
##                                             scenario   mean_change
## Felis_catus_25percent               Felis_catus_25percent  7.114727e-05
## Felis_catus_50percent               Felis_catus_50percent  1.790289e-04
## Felis_catus_75percent               Felis_catus_75percent  3.637063e-04
## Felis_catus_90percent               Felis_catus_90percent  5.521448e-04
## Felis_catus_100percent             Felis_catus_100percent  7.409483e-04
## Rattus_spp_25percent                 Rattus_spp_25percent -4.089453e-04
## Rattus_spp_50percent                 Rattus_spp_50percent -5.010805e-04
## Rattus_spp_75percent                 Rattus_spp_75percent -5.383842e-04
## Rattus_spp_90percent                 Rattus_spp_90percent -5.918767e-04
## Rattus_spp_100percent               Rattus_spp_100percent -8.361968e-04
## Salvator_merianae_25percent   Salvator_merianae_25percent  1.899691e-05
## Salvator_merianae_50percent   Salvator_merianae_50percent  5.986138e-05
## Salvator_merianae_75percent   Salvator_merianae_75percent  1.617475e-04
## Salvator_merianae_90percent   Salvator_merianae_90percent  3.658430e-04
## Salvator_merianae_100percent Salvator_merianae_100percent  2.214027e-03
## Rhinella_25percent                   Rhinella_25percent -6.290717e-04
## Rhinella_50percent                   Rhinella_50percent -7.882099e-04
## Rhinella_75percent                   Rhinella_75percent -8.566811e-04
## Rhinella_90percent                   Rhinella_90percent -8.645310e-04
## Rhinella_100percent                 Rhinella_100percent -3.932737e-04
## Cats_and_Rats                             Cats_and_Rats  1.321277e-03
## Cats_and_Tegu                             Cats_and_Tegu  3.206618e-03
## All_Invasive                               All_Invasive -9.681715e-04
```

```
##                               positive_species negative_species sign_changes
## Felis_catus_25percent                        5               10            9
## Felis_catus_50percent                        5               10           10
## Felis_catus_75percent                        5               10           10
## Felis_catus_90percent                        5               10           10
## Felis_catus_100percent                       5               10           10
## Rattus_spp_25percent                         5                9            4
## Rattus_spp_50percent                         5               10           17
## Rattus_spp_75percent                         5               10           35
## Rattus_spp_90percent                         4               11           45
## Rattus_spp_100percent                        4               11           45
## Salvator_merianae_25percent                  7                8           13
## Salvator_merianae_50percent                  8                7           14
## Salvator_merianae_75percent                  8                7           14
## Salvator_merianae_90percent                  7                8           14
## Salvator_merianae_100percent                 7                8           15
## Rhinella_25percent                           4               10           19
## Rhinella_50percent                           5               10           40
## Rhinella_75percent                           5               10           44
## Rhinella_90percent                           5               10           44
## Rhinella_100percent                          5               10           45
## Cats_and_Rats                                5               10           45
## Cats_and_Tegu                                9                6           17
## All_Invasive                                 5                7           53
##                                 Trachylepis   Amphisbaena       Kerodon
## Felis_catus_25percent       -0.0004214203 -2.502687e-06 -0.0001805759
## Felis_catus_50percent       -0.0013515255 -5.566893e-06 -0.0004771737
## Felis_catus_75percent       -0.0033891649 -9.402392e-06 -0.0009957878
## Felis_catus_90percent       -0.0057686875 -1.217625e-05 -0.0015163757
## Felis_catus_100percent      -0.0083186540 -1.423944e-05 -0.0020257753
## Rattus_spp_25percent         0.0004000356 -5.773626e-06 -0.0013037016
## Rattus_spp_50percent         0.0006113935 -7.944137e-06 -0.0016642127
## Rattus_spp_75percent         0.0009777767 -9.818743e-06 -0.0018474827
## Rattus_spp_90percent         0.0015314192 -1.207353e-05 -0.0019458061
## Rattus_spp_100percent        0.0030527294 -1.822359e-05 -0.0020857406
## Salvator_merianae_25percent -0.0002862363  8.164202e-06  0.0001228423
## Salvator_merianae_50percent -0.0007413115  2.633236e-05  0.0002360773
## Salvator_merianae_75percent -0.0015130315  8.566534e-05  0.0003380060
## Salvator_merianae_90percent -0.0022977364  2.577949e-04  0.0003898622
## Salvator_merianae_100percent -0.0039961651 2.268709e-03  0.0003760220
## Rhinella_25percent          -0.0067567653 -3.813041e-06 -0.0012618861
## Rhinella_50percent          -0.0085065496 -5.675902e-06 -0.0016173126
## Rhinella_75percent          -0.0093246640 -6.824461e-06 -0.0017857024
## Rhinella_90percent          -0.0096510413 -7.461317e-06 -0.0018510859
## Rhinella_100percent         -0.0099470365 -9.315237e-06 -0.0018964633
## Cats_and_Rats               -0.0107621418 -1.975802e-05 -0.0023481906
## Cats_and_Tegu               -0.0076325642  9.416272e-04 -0.0001548564
## All_Invasive                -0.0110008395 -6.474975e-06 -0.0021856406
```

Per-scenario heatmaps of change relative to baseline

Managed (removed/reduced) species are excluded from each heatmap, since their own effect values are not meaningful once they are being removed from the system.

```r
management_heatmaps <- list()

for (name in names(management_results)) {

  delta_matrix <- management_delta_effect_matrices[[name]]

  managed_species <- unique(management_scenarios[[name]]$source)
  delta_matrix <- delta_matrix[
    !(rownames(delta_matrix) %in% managed_species),
    !(colnames(delta_matrix) %in% managed_species),
    drop = FALSE
  ]
  diag(delta_matrix) <- NA

  df <- reshape2::melt(delta_matrix)
  colnames(df) <- c("Receiver", "Source", "Effect")

  management_heatmaps[[name]] <- ggplot(df, aes(Source, Receiver, fill = Effect)) +
    geom_tile() +
    scale_fill_gradient2(
      low = "#762A83", mid = "white", high = "#1B7837",
      midpoint = 0, limits = c(-max_delta_effect, max_delta_effect)
    ) +
    coord_fixed() +
    theme_bw() +
    theme(
      axis.text.x = element_text(angle = 90, hjust = 1, size = 8),
      axis.text.y = element_text(size = 8)
    ) +
    labs(
      title = paste("Change relative to baseline:", name),
      x = "Effect source",
      y = "Effect receiver",
      fill = expression(Delta * " total effect")
    )
}
```

Per-scenario species-response bar plots

Axes are standardized across all scenarios (`max_species_change`) so scenarios can be visually compared for effect magnitude.

```r
management_species_plots <- list()

for (name in names(management_results)) {

  df <- data.frame(
    species = names(management_results[[name]]$delta_mean_total_effect),
    delta = as.numeric(management_results[[name]]$delta_mean_total_effect)
  )

  managed_species <- management_scenarios[[name]]$source
```

```
  df <- df[!(df$species %in% managed_species), ]
  df <- df[order(df$delta), ]

  management_species_plots[[name]] <- ggplot(
    df, aes(x = reorder(species, delta), y = delta, fill = delta > 0)
  ) +
    geom_col() +
    coord_flip() +
    scale_y_continuous(limits = c(-max_species_change, max_species_change)) +
    scale_fill_manual(values = c("#B2182B", "#1B7837"), guide = "none") +
    geom_hline(yintercept = 0, colour = "black") +
    labs(
      title = paste("Species responses:", name),
      subtitle = "Axis standardized across all management scenarios",
      x = NULL,
      y = expression(Delta * " mean total effect")
    ) +
    theme_bw()
}
```

Overall ecosystem-level response summary

A single summary figure comparing the mean network-wide change in total effect across all management scenarios.

```
plot_management_summary <- ggplot(
  management_summary2,
  aes(x = reorder(scenario, mean_change), y = mean_change, fill = mean_change > 0)
) +
  geom_col() +
  coord_flip() +
  scale_y_continuous(limits = c(
    -max(abs(management_summary2$mean_change)),
    max(abs(management_summary2$mean_change))
  )) +
  scale_fill_manual(values = c("#B2182B", "#1B7837"), guide = "none") +
  geom_hline(yintercept = 0) +
  labs(title = "Overall ecosystem response", x = NULL, y = expression(Delta * " mean total effect")) +
  theme_bw()

plot_management_summary
```

Part VII — Save outputs

All figures, summary tables, and full result objects (as `.rds` files, for re-loading into R without re-running the whole pipeline) are written to the `outputs/` folder.

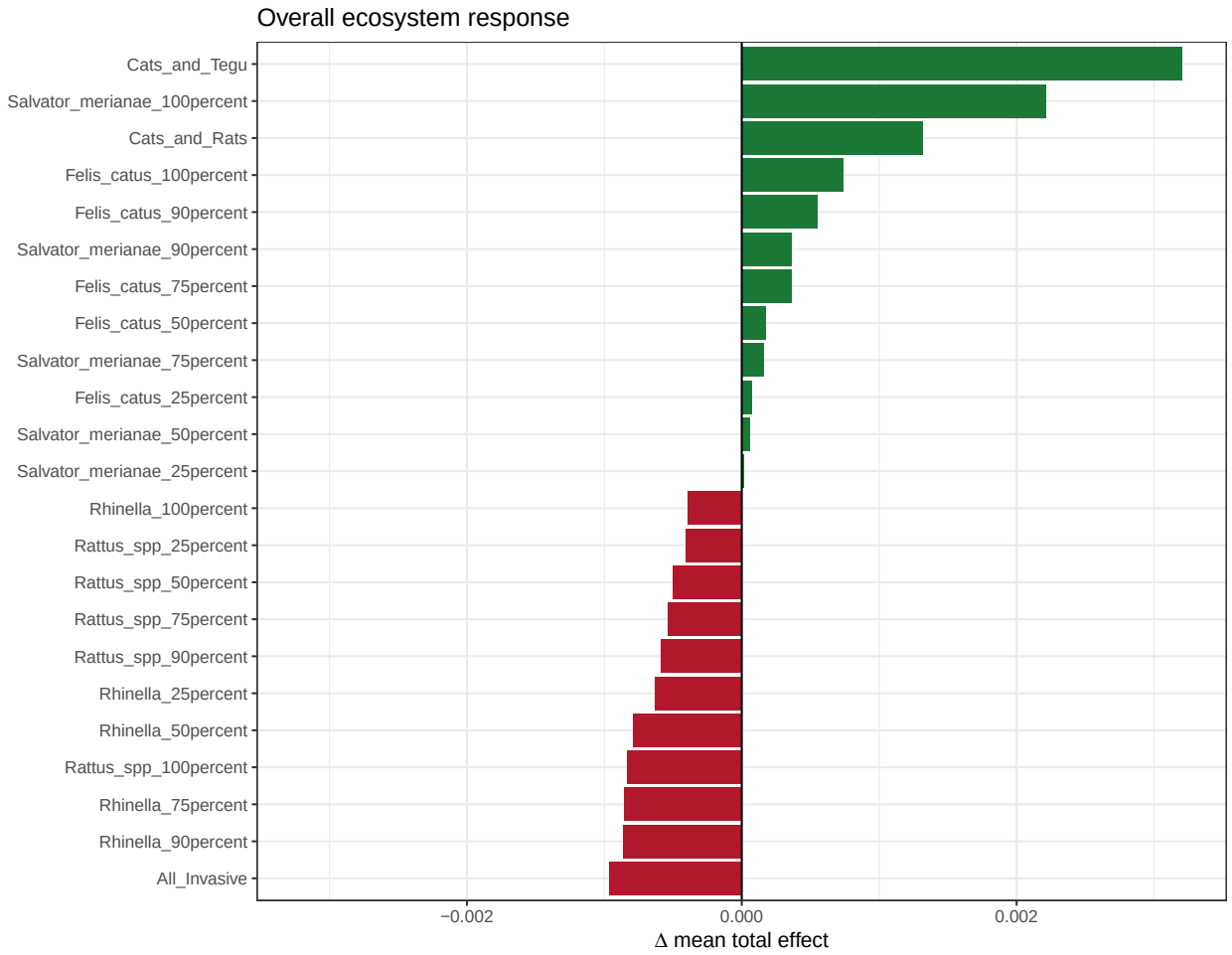

Figure 7: Overall predicted ecosystem-wide response (mean total effect across all species) for each management scenario.

Save main figures

```
ggsave("outputs/Figure_01_Foodweb.png", foodweb_plot, width = 10, height = 8, dpi = 300)
ggsave("outputs/Figure_02_DirectEffectsHeatmap.png", direct_effect_heatmap, width = 10, height = 9, dpi
ggsave("outputs/Figure_03_TotalEffectsHeatmap.png", total_effect_heatmap, width = 10, height = 9, dpi =
ggsave("outputs/Figure_04_SensitivityHistogram.png", response_strength_sensitivity_plot, width = 8, hei
ggsave("outputs/Figure_09_TopSpeciesStability.png", plot_top5_stability, width = 8, height = 5, dpi = 3
ggsave("outputs/Figure_10_Management_summary.png", plot_management_summary, width = 8, height = 6, dpi =
```

Save management-scenario heatmaps and species-response plots

```
for (name in names(management_heatmaps)) {
  ggsave(
    filename = file.path("outputs", paste0("Management_", gsub("[^A-Za-z0-9]", "_", name), ".png")),
    plot = management_heatmaps[[name]],
    width = 9, height = 8, dpi = 300
  )
}

for (name in names(management_species_plots)) {
  ggsave(
    filename = file.path("outputs", paste0("SpeciesResponse_", gsub("[^A-Za-z0-9]", "_", name), ".png"))
    plot = management_species_plots[[name]],
    width = 8, height = 6, dpi = 300
  )
}
```

Save tables

```
write.csv(direct_effect_matrix, "outputs/direct_effect_matrix.csv", row.names = TRUE)
write.csv(total_effect_matrix, "outputs/total_effect_matrix.csv", row.names = TRUE)
write.csv(effects_Matrix, "outputs/effects_input_matrix.csv", row.names = TRUE)
write.csv(sensitivity_summary, "outputs/sensitivity_summary.csv", row.names = FALSE)
write.csv(management_summary, "outputs/management_summary.csv", row.names = FALSE)
write.csv(management_sign_changes, "outputs/management_sign_changes.csv", row.names = FALSE)
write.csv(management_endemic_species, "outputs/management_endemic_species.csv", row.names = FALSE)
write.csv(management_summary2, "outputs/management_summary2.csv", row.names = FALSE)
```

Save complete result objects

```
saveRDS(effect_results, "outputs/effect_results.rds")
saveRDS(management_results, "outputs/management_results.rds")
saveRDS(management_summary, "outputs/management_summary.rds")
saveRDS(sensitivity_summary, "outputs/sensitivity_summary.rds")

saveRDS(management_direct_effect_matrices, "outputs/management_direct_effect_matrices.rds")
```

```r
saveRDS(management_total_effect_matrices, "outputs/management_total_effect_matrices.rds")
saveRDS(management_delta_effect_matrices, "outputs/management_delta_effect_matrices.rds")

cat("\nOutputs successfully saved to 'outputs/'\n")
```

```
##
## Outputs successfully saved to 'outputs/'
```

Session info

Included for reproducibility — records exact package versions used to generate this document.

```r
sessionInfo()
```

```
## R version 4.6.0 (2026-04-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: Europe/Berlin
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] reshape2_1.4.5     ggraph_2.2.2       ggplot2_4.0.3      igraph_2.3.1
## [5] data.table_1.18.4  readxl_1.5.0       Require_2.0.0.9000
##
## loaded via a namespace (and not attached):
##  [1] viridis_0.6.5      generics_0.1.4     tidyr_1.3.2        stringi_1.8.7
##  [5] digest_0.6.39      magrittr_2.0.5     evaluate_1.0.5     grid_4.6.0
##  [9] RColorBrewer_1.1-3 fastmap_1.2.0      plyr_1.8.9         cellranger_1.1.0
## [13] ggrepel_0.9.8      gridExtra_2.3      purrr_1.2.2        viridisLite_0.4.3
## [17] scales_1.4.0       tweenr_2.0.3       cli_3.6.6          rlang_1.2.0
## [21] graphlayouts_1.2.4 polyclip_1.10-7    pak_0.9.5          tidygraph_1.3.1
## [25] withr_3.0.2        cachem_1.1.0       yaml_2.3.12        otel_0.2.0
## [29] tools_4.6.0        parallel_4.6.0     memoise_2.0.1      dplyr_1.2.1
## [33] vctrs_0.7.3        R6_2.6.1           lifecycle_1.0.5    stringr_1.6.0
## [37] MASS_7.3-65        pkgconfig_2.0.3    pillar_1.11.1      gtable_0.3.6
## [41] glue_1.8.1         Rcpp_1.1.1-1.1     ggforce_0.5.0      xfun_0.57
## [45] tibble_3.3.1       tidyselect_1.2.1   rstudioapi_0.18.0  sys_3.4.3
```

```
## [49] knitr_1.51         farver_2.1.2       htmltools_0.5.9    labeling_0.4.3
## [53] rmarkdown_2.31     compiler_4.6.0     S7_0.2.2
```